

Astria Backend Mars Image Sampler Service

Marsette Vona <marsette.a.vona@jpl.nasa.gov>

August 18, 2026

Contents

1	Introduction	3
2	Building	3
3	Explicit Sample Request	4
4	Explicit Sample Response	5
5	Spatial Volume Request	5
6	Spatial Volume Response	6
7	POST Requests	7
8	AWS Credentials	8
9	Configuration	8

1 Introduction

The Mars Image Sampler (`mis_rest_service`) Tomcat servlet returns data for individual pixels of VICAR images stored on S3, any HTTP[S] server, or the local filesystem. Also supports images with PDS/ODL wrapper (extension `.IMG` or `.VIC`). Optionally labels and unpacks the pixel data. Uses partial reads so that only the headers and the data for the requested pixel are downloaded.

Only band sequential VICAR data is supported (`ORG=BSQ`). Band interleaved by line (`ORG=BIL`) and band interleaved by pixel (`ORG=BIP`) formats are not supported. All VICAR data types except `COMP` (complex number) and `VAX` floating point are supported.

The servlet builds as `image_sampler.war`. Also builds as `image_sampler.jar` which can be used as a library in other applications, or can be run as a stand-alone command line tool. Run `java -jar image_sampler.jar -h` for a list of available options.

For backwards compatibility the service URL can optionally end in `/get`.

A request like `https://SERVER/image_sampler/version` will return the deployed version.

All requests must specify an RDR file to query with a parameter like `url=SRC` where `SRC` is a URL to the source PDS or VICAR image to sample. It may be an `s3://`, `http[s]://`, or `file://` URL or a local path in one of the following forms:

- `s3://BUCKET/KEY`
- `http[s]://BUCKET.s3-REGION.amazonaws.com/KEY`
- `http[s]://BUCKET.s3.REGION.amazonaws.com/KEY`
- `http[s]://BUCKET.s3.amazonaws.com/KEY`
- `http[s]://s3-REGION.amazonaws.com/BUCKET/KEY`
- `http[s]://s3.amazonaws.com/BUCKET/KEY`
- `http[s]://PATH`
- `file://PATH`
- `PATH`.

If `REGION` is included and differs from the region of the AWS credentials (see below) the query will likely fail. `..` is disallowed in `KEY` and `PATH`.

2 Building

1. Install latest **JDK 21**. Make sure the environment variable `JAVA_HOME` is set properly, e.g. `$JAVA_HOME/bin/java` should be the java runtime executable. Also make sure that `javac` and `java` are added to `PATH`.
2. Install latest **Maven**. Ensure the `mvn` executable is added to your `PATH`.
3. Configure Maven in `~/.m2/settings.xml` as necessary to access the dependency repositories.

4. `mvn package` should build `target/image_sampler.jar`.
5. `mvn -P war-FLAVOR package` should build `target/image_sampler.war`, where `FLAVOR` is one of `docker`, `localhost`, or `localhost-noaws`.
6. All build files are created under the relative directory `target/`, which is git ignored. Recursively delete it to start fresh (though you may still have cached maven dependencies).

3 Explicit Sample Request

To explicitly request image samples at one or more pixel coordinates, issue a request like:

```
https://SERVER/image_sampler[/get]?url=SRC&line=LINE&sample=SAMPLE
    [&batch=true] [&origin=BASE] [&label=true] [&rdr=TYPE] [&interp=MODE]
```

`LINE` is the line (i.e. row) number of the pixel to get. Any fractional part will be discarded unless `interp=MODE` is specified with `MODE` different from `none`. For backwards compatibility the first (topmost) line is 1. Add `origin=0` for zero-based indexing. `LINE` can also be a comma separated list of numbers, more than one forces batch mode, length must equal `SAMPLE`.

`SAMPLE` is the sample (i.e. column) number of the pixel to get. Any fractional part will be discarded unless `interp=MODE` is specified with `MODE` different from `none`. For backwards compatibility the first (leftmost) sample is 1. Add `origin=0` for zero-based indexing. `SAMPLE` can also be a comma separated list of numbers, more than one forces batch mode, length must equal `LINE`.

If `batch=true` is present then the return is always a JSON array of JSON objects, even if only one pixel is requested.

If `origin=BASE` is present `BASE` must be either 0 or 1, which sets the first line and sample index. The default is 1 for backward compatibility.

If `label=true` is present then the response is a JSON dictionary with the pixel data labeled and unpacked. There will be one entry per band. The key of the entry is a string giving the name of the band. The value of the entry may be an integer, a floating point number, or a string. For image types which support labelling but not unpacking, numeric band digital number (DN) values are copied directly from the source image. String values result from unpacking band data from some types of original source images (e.g. reachability, roughness, goodness, etc).

With `label=true` the entries in the returned JSON dictionary may come in any order. For consistent presentation the consuming application may choose to order them, e.g. in alphanumeric order of the key names.

If `label=true` is absent (or `label=false` is present) the response is a JSON array with one number per band (in order) giving the DN values of the requested

pixel. The numbers will either be all integers or all floating point, depending on the format of the source image.

If `rdr=TYPE` is present and not `rdr=auto` it overrides inferring the mission-specific product type from the source product ID. Must match an `image_type` id attribute in marsviewer `image_config.xml`. This is supported so that the REST service can be mission independent. The client may retrieve `TYPE` e.g. from OCS where it may have been stored using mission specific ingestion.

If `interp=MODE` is present then `MODE` must be `none`, `interp_dn` or `interp_nonzero`. The default behavior is the same as `interp=none`: no interpolation is performed, and any fractional parts of `LINE` and `SAMPLE` are discarded. With `interp=interp_dn` 2x2 bilinear interpolation is performed without hole filling. With `interp=interp_nonzero` then hole filling is attempted by replacing zero pixels in the 2x2 neighborhood with a symmetric average of their 8-neighbors where possible (otherwise 0 is returned). Interpolation is always performed separately per-band. Combining `label=true` with interpolation can give undefined results for product types which involve unpacking.

4 Explicit Sample Response

Explicit sample pixel data (“digital numbers”, or DNs) is always returned in JSON form. If `label=true` then each pixel is “unpacked” into a JSON object with metadata about the pixel value. Otherwise, each pixel is returned as a JSON array of numbers, except not-a-number and infinite values are returned as strings, e.g. NaN, [-]Infinity.

If only one pixel is requested and batch mode is not explicitly requested, the return is just the JSON for that pixel.

Otherwise, the return is a JSON array of objects, one per pixel. Each object has four fields: `line`, `sample`, `origin`, and `data`. The `data` field contains the pixel value data. The `line` and `sample` fields are the requested coordinates of the pixel, zero-based if `origin=0` and one-based if `origin='1`.

5 Spatial Volume Request

Convex volumes of samples can also be requested when the underlying RDR has 3 bands that can be interpreted as X (band 0), Y (band 1), and Z (band 2), such as XYZ and XYM for Mars surface missions. The convex volume is bounded by planes and/or spheres in a query like this:

```
https://SERVER/image_sampler[/get]?url=SRC[&plane=PLANE][&plane=PLANE...]  
[&sphere=SPHERE][&sphere=SPHERE...] [&surface=PLANE]
```

`PLANE` is a comma separated list of 4 floating point numbers `nx,ny,nz,d` (no scientific notation) which define an outward pointing unit normal `N=(nx,ny,nz)`

and perpendicular distance d from the origin. Returned points Q are on or below the plane such that $Q \cdot N \leq d$. The coordinate frame and units of N and d is the same as the data in the SRC RDR.

SPHERE is a comma separated list of 4 floating point numbers cx, cy, cz, r (no scientific notation) which define a center $C=(cx, cy, cz)$ and radius r . Returned points Q are on or in the sphere such that $||Q - C|| \leq r$. The coordinate frame and units of C and r is the same as the data in the SRC RDR.

At least one sphere or plane must be specified. Any combination of multiple spheres and/or planes may be specified; the returned data is in the intersection of them all.

Input points with $(x,y,z) = (0,0,0)$ are considered invalid and are ignored.

See below regarding the **surface** option.

6 Spatial Volume Response

If **surface=PLANE** is absent then the returned data is a JSON array of the points within the query volume. Each point is an array of three numbers $[x,y,z]$ read directly from the SRC RDR (and thus in its coordinate frame and units). If no points were within the volume then an empty array is returned. In this case up to the total number of pixels in the SRC RDR can be returned if the request volume is sufficiently large. The server will stream the response out incrementally and callers should be prepared to receive a potentially large amount of data.

When **surface=PLANE** is present the server reads the entire SRC RDR before sending the response. This may take some time depending on the size of the RDR. In this case the returned data is a JSON object giving statistics of the query population distances to that plane, where the distance units are the same as the units of the data in the SRC RDR:

```
{
  num_points: Number,
  min_distance: Number,
  max_distance: Number,
  histogram_bin_width: Number,
  histogram_outliers_below: Number,
  histogram_outliers_above: Number,
  histogram_below: Number[]
  histogram_above: Number[]
}
```

num_points is the number of SRC RDR points the volume includes.

min_distance is the minimum signed distance from any included point to the surface plane, or 0 if **num_points**=0.

max_distance is the maximum signed distance from any included point to the surface plane, or 0 if **num_points**=0.

histogram_bin_width is the width of each histogram bin, set in the server configuration.

histogram_outliers_below is the number of included points below the bottom of the lowest histogram bin.

histogram_outliers_above is the number of included points at or above the top of the highest histogram bin.

histogram_below is the histogram of points below the surface plane. It is an array of $m \leq n$ bins of width w where n and w are set in the server configuration. Typically n defaults to 10000 and w defaults to 0.001. For $0 \leq i < m$ **histogram_below**[i] is the number of included points with signed distance d in the range $-(i + 1) * w \leq d < -i * w$ to the plane. For $m \leq i < n$ the number of included points in that range is 0. **histogram_outliers_below** can only be nonzero when $m == n$, it serves to indicate overflow.

histogram_above similar to **histogram_below** but for points on above the surface plane. **histogram_above**[i] is the number of included points with signed distance d in the range $i * w \leq d < (i + 1) * w$ to the plane.

7 POST Requests

POST requests are also accepted. The POST payload must be a JSON object with the following fields. **line**[**s**]/**sample**[**s**] and **planes**/**spheres** are mutually exclusive and indicate either an explicit sample request or a spatial volume request.

rdr_url (string, required) - same as GET parameter **url**.

rdr_type (string, default auto) - same as GET parameter **rdr**.

line or **lines** (number or number array, respectively, required for explicit sample requests) - same as GET parameter **line**.

sample or **samples** (number or number array, respectively, required for explicit sample requests) - same as GET parameter **sample**.

origin (int, default 1) - same as eponymous GET parameter.

batch (bool, default false) - same as eponymous GET parameter.

label (bool, default false) - same as eponymous GET parameter.

interp (string, default none) - same as eponymous GET parameter.

planes (array of arrays of 4 numbers, optional, only for volume requests) - same as GET parameter **plane**.

spheres (array of arrays of 4 numbers, optional, only for volume requests) - same as GET parameter **sphere**.

surface (array of 4 numbers, optional, only for volume requests) - same as eponymous GET parameter.

8 AWS Credentials

The AWS credentials used to access S3 are typically taken from the request if the **X-AWS** header is present. If so, it is base 64 decoded, parsed as JSON, and the following entries are used: **id**, **secret**, **session**, **region**.

If the **X-AWS** header is not present or the server is configured to ignore that header then the server uses the same AWS credentials for all queries. Those credentials may come from a profile in the AWS credentials file on the server or from the default AWS credentials on the server (e.g. environment variables, IAM role).

AWS credentials are optional and only need to be configured or provided if the service needs to directly access S3.

9 Configuration

The following servlet initialization parameters may be specified in **web.xml** for the REST service. Uppercase versions of the same values are also recognized as environment variables which are used in Docker deployments of the REST service.

Note on default values: the defaults described here refer to the defaults in the code. They can be overridden on a per-venue basis.

- **debug_sampler**: Defaults to false. Set to **true** to enable extra logging.
- **enable_s3**: Must be **true** or **false**. Defaults to **true**. Enables reading from S3.
- **enable_http**: Must be **true**, **false**, or **auto**. Defaults to **true**. Enables reading inputs from http[s] URLs.
- **rdr_url_whitelist_patterns**: Defaults to empty, meaning allow all. Comma separated list of URL regex to whitelist RDR URLs. If this is empty no whitelist is applied, but the **enable_s3** and **enable_http** settings are still respected. This whitelist is for security and cannot be disabled.
- **fs_input_dir**: optional content root for filesystem input images. Applies only to REST service. Empty or null disables filesystem input for REST service (command line interface will still allow local paths).
- **mission**: one of **MSL**, **M20**, **M2020**, or **CADRE**. Defaults to **M20**. Used to determine the format for parsing input filenames as mission product IDs when **rdr=TYPE** is absent.

- **aws_profile**: Defaults to `null`. AWS credentials profile name if not using credentials from X-AWS request header. Use `null` for the **default credential provider chain**.
- **aws_region**: Defaults to `us-gov-west-1`. AWS region name. Use `null` for to use the **default region provider chain**.
- **use_aws_header**: one of `always`, `never`, or `prefer`. Defaults to `prefer`. Whether to use the X-AWS request header. In `prefer` mode the X-AWS header is used if present, otherwise **aws_profile** and **aws_region** are used. In `always` mode the X-AWS header is always used and the request fails if it's absent. In `never` mode the X-AWS header is ignored and **aws_profile** and **aws_region** are always used.
- **cache_s3_credentials**: Defaults to 64. When using X-AWS request headers for S3 credentials, cache up to this many credentials for potential near term re-use. Only applies to REST service.
- **max_s3_credentials_age**: Defaults to 900 (15 minutes). When using X-AWS request headers for S3 credentials, cache credentials for potential near term re-use up to this many seconds from when they were created. Only applies to REST service.
- **ignore_subdirs**: Defaults to empty. Comma separated list of subdirectories to exclude from processing, e.g. `browse,ids-pipeline,orbital`. If any path segment of a request URL matches any ignored subdir the request will fail.
- **sampler_max_concurrent_requests**: Defaults to -1 (unlimited). If positive then limit the number of concurrent requests. Additional load will result in failed requests.
- **surface_histogram_bin_width**: Defaults to 0.001. Bin width in the histogram returned by spatial volume queries with **surface=PLANE**.
- **surface_histogram_max_bins**: Defaults to 10000. Maximum number of bins in each half of the histogram returned by spatial volume queries with **surface=PLANE**.
- **max_cached_samples**: Defaults to 1000000. Maximum number of samples to cache. Here a “sample” is represented as an 8 byte double precision DN value for a given line, sample, and band. Each query that the server is currently serving has a separate LRU sample cache. Sample caching can be helpful for explicit batch queries, particularly if interpolation is enabled and adjacent pixels are requested. Sample caching is not used for volume queries.
- **max_cached_record_bytes**: Defaults to 10000000. Maximum number of bytes to cache for full records. Here a “record” is an array of the raw DN values for one line of one band. E.g. if the image width is 1000 and there are 2 bytes per sample then the default setting will allow up to $10000000 / 2000 = 5000$ records to be cached, or $5000 / 3 = 1666$ full lines of pixels for a 3 band image. Each query that the server is currently serving has a separate LRU record cache. Record caching can be helpful for batch queries on a compressed source RDR because VICAR compression is per-line and the cached records contain the decompressed data. Record

caching is also used, if enabled, for all volume queries. In that case the entire record cache is repeatedly filled with the maximum possible number of lines from the source file to minimize the number of S3 read transactions, since the whole file must be read for each volume query.